

6.4.3 Security and Privacy Limitations

Notification action tokens are hashed, and state changes require POST after confirmation. Access tokens are stored in frontend sessionStorage rather than persistent localStorage. Local CV storage lacks documented malware scanning, encryption, retention enforcement, and recovery testing. Public Swagger and Prometheus access are suitable for local demonstration but should be restricted in production. Privacy, fairness, and management of users' personal data have not been validated with a real deployment population.

6.4.4 Reliability and Maintainability Limitations

The final remediation pass resolved the observed optimistic-lock benchmark exception, scheduler and configuration drift, mock Job fallback values, persistent localStorage authentication, the oversized frontend bundle warning, and missing E2E Job cleanup. Audit records are still written directly by multiple services, and the evaluated working tree is not a fixed release commit. These limits make production operation and exact reproduction of the results more difficult.

Table 6.3. Principal limitations and impact

Limitation	Impact on conclusions
Synthetic benchmark	Supports causal behavior only, not real hiring effectiveness
Lexical representation	Limits synonym, semantic, and context understanding
Production concurrency not evaluated	Final benchmark is clean, but production conflict and retry behavior remain unproven
Local-only health verification	Health passed locally but does not establish deployed availability
Chromium-only 20-test automation suite	Limits cross-browser and real-user usability conclusions
Email delivery and abuse controls not production-tested	Hashed confirm-then-POST flow is implemented; rate limiting and real delivery remain unverified
Small localhost latency sample	Cannot establish scale, throughput, or cloud performance
Dirty worktree and mutable E2E database	Weakens exact experiment reproduction

6.5 Future Work

Future security work should add rate limiting, stronger session handling, protected management endpoints, malware scanning, encrypted CV storage, retention rules, and tested backup recovery. Email delivery should be tested with a real provider, including expiry, replay, scanner behavior, and failure recovery. CI should continue checking background logs and asynchronous completion rather than relying only on foreground test status.

The matching baseline can then be extended using a hybrid approach. Domain-aware tokenization and aliases should first address punctuation-sensitive technologies and Vietnamese phrases. Sparse lexical scores can be combined with sentence or skill embeddings and structured constraints. Any new model should be evaluated against the current baseline on a frozen, independently labeled dataset rather than assumed superior because it is more complex.

Evaluation should expand to recruiter-labeled CV–JD pairs, temporal holdout sets, inter-rater agreement, hard-negative analysis, subgroup/fairness checks, and calibration of labels. A controlled user study should measure task completion, time, comprehension of explanations, trust calibration, and ability to override automation. Cross-browser E2E, load testing, failure injection, backup/restore, email delivery, and security testing should be included in release evidence.

Operational development should move CVs to protected object storage, define encryption and retention policy, centralize audit generation and redaction, strengthen token/session architecture, and associate each release with a clean commit and a fixed evidence archive. Integration with external ATS or Job boards should occur only after consent, ownership, error recovery, and audit contracts are defined.

6.6 Conclusion

CareerFit demonstrates controlled IT recruitment automation by connecting job discovery, explainable matching, profile-based recommendations, feedback learning, policy-driven actions, and audit records. It keeps scores, business state, and user decisions visible.

Local evaluation passed the backend tests, frontend build, selected browser workflows, health checks, and controlled Rocchio benchmark. This supports a tested Human-in-the-Loop workflow, not a system ready for real hiring decisions.

REFERENCES

- [1] G. Salton, A. Wong, and C. S. Yang, “A vector space model for automatic indexing,” *Communications of the ACM*, vol. 18, no. 11, pp. 613–620, 1975, doi: 10.1145/361219.361220.
- [2] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [3] C. Buck and P. Koehn, “Quick and reliable document alignment via TF/IDF-weighted cosine distance,” in *Proceedings of the First Conference on Machine Translation*, vol. 2, Berlin, Germany, 2016, pp. 672–678, doi: 10.18653/v1/W16-2365.
- [4] J. J. Rocchio, “Relevance feedback in information retrieval,” in *The SMART Retrieval System: Experiments in Automatic Document Processing*, G. Salton, Ed. Englewood Cliffs, NJ, USA: Prentice-Hall, 1971, pp. 313–323.
- [5] K. Järvelin and J. Kekäläinen, “Cumulated gain-based evaluation of IR techniques,” *ACM Transactions on Information Systems*, vol. 20, no. 4, pp. 422–446, 2002, doi: 10.1145/582415.582418.
- [6] N. Drushchak and M. Romanyshyn, “Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings,” in *Proceedings of the Third Ukrainian Natural Language Processing Workshop*, Torino, Italy, 2024, pp. 8–13, doi: 10.63317/2dcvy45ws6yb.
- [7] X. Yu, R. Xu, C. Xue, J. Zhang, X. Ma, and Z. Yu, “ConFit v2: Improving resume-job matching using hypothetical resume embedding and runner-up hard-negative mining,” in *Findings of the Association for Computational Linguistics: ACL 2025*, Vienna, Austria, 2025, pp. 12775–12790, doi: 10.18653/v1/2025.findings-acl.661.
- [8] S. Vaishampayan et al., “Human and LLM-based resume matching: An observational study,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, Albuquerque, NM, USA, 2025, pp. 4823–4838, doi: 10.18653/v1/2025.findings-naacl.270.
- [9] E. Tabassi, *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*, NIST AI 100-1. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2023, doi: 10.6028/NIST.AI.100-1.
- [10] M. Raghavan, S. Barocas, J. Kleinberg, and K. Levy, “Mitigating bias in algorithmic hiring: Evaluating claims and practices,” in *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, Barcelona, Spain, 2020, pp. 469–481, doi: 10.1145/3351095.3372828.
- [11] R. T. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, University of California, Irvine, CA, USA, 2000.

- [12] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Internet Engineering Task Force, RFC 7519, May 2015, doi: 10.17487/RFC7519.
- [13] K. Kent and M. Souppaya, Guide to Computer Security Log Management, NIST SP 800-92. Gaithersburg, MD, USA: National Institute of Standards and Technology, 2006, doi: 10.6028/NIST.SP.800-92.
- [14] A. Colas, J. Araki, Z. Zhou, B. Wang, and Z. Feng, “Knowledge-grounded natural language recommendation explanation,” in Proceedings of the 6th BlackboxNLP Workshop, Singapore, 2023, pp. 1–15, doi: 10.18653/v1/2023.blackboxnlp-1.1.

APPENDICES

Appendix A. Requirement–Test Traceability Matrix

The core traceability chain is: authentication and role controls → SecurityHardeningTest and P0 login flows; CV ingestion → service and integration tests plus the Candidate upload flow; matching and feedback → AlgorithmEvaluatorTest; Job lifecycle → recruiter P0 create/verify/delete flow; administration → suspend/activate P0 flow; operations → Actuator health and build evidence.

Appendix B. Selected API Contracts

Representative endpoints are POST /api/auth/login; GET and POST /api/jobs; POST /api/cvs/upload; GET /api/matchings; POST /api/feedback; GET and PATCH /api/automation/policies/me; GET then POST /api/email-action/redeem; and GET /actuator/health. Protected endpoints require a signed JWT and enforce role or ownership rules in the backend.

Appendix C. Data Model Summary

Identity data is centered on Account and role-specific profiles. Recruitment data includes Job, CV, Matching, Application, Invitation, and Feedback. Automation and operations use AutomationPolicy, EmailAction, EmailToken, Notification, AuditLog, and scheduler state. Flyway migrations provide the reproducible schema history; action tokens are persisted as SHA-256 hashes.

Appendix D. Evaluation Summary

The final evidence package contains the complete backend test log, isolated algorithm benchmark, frontend production build, browser-based P0 results, runtime health response, screenshots, and evaluation/result.json. Chapter 5 reports the exact observed results and limitations; these artifacts support demo and thesis-defense readiness rather than a production-deployment claim.

Appendix E. UAT and Demonstration Script

1) Search for a public IT Job and open its details. 2) Sign in as Candidate, upload/select a CV, inspect ranked matches and explanations, apply, and withdraw. 3) Sign in as Recruiter, create a Job, inspect candidate ranking, and remove test data. 4) Configure AutoFit and explain threshold, quota, cooldown, and human override. 5) Sign in as Administrator, suspend and reactivate a test account, then inspect the audit view. 6) Show Actuator health and the archived test evidence.

Appendix F. Local Deployment Instructions

Prerequisites are Java 21, Node.js with npm, Docker Desktop, and Git. Start PostgreSQL with docker compose up -d postgres. From Backend/careerfit-backend, run mvnw.cmd spring-boot:run. From Frontend, run npm install and npm run dev. Host-mode PostgreSQL is localhost:5433; services inside the Compose network use postgres:5432. Verify GET <http://localhost:8080/actuator/health> and open <http://localhost:5173>. Secrets and production controls must be supplied through environment-specific configuration.